

Der wichtigste Punkt zuerst: **Ein Unity-Projekt hat normalerweise keine zentrale Projektdatei wie eine `.csproj`, die das Projekt definiert.** Unity erkennt ein Projekt im Wesentlichen daran, dass im Root-Ordner bestimmte Verzeichnisse liegen – vor allem `Assets/`, `Packages/` und `ProjectSettings/`.

Wenn du als Programmierer draufguckst, ist dieses mentale Modell brauchbar:

```
MyUnityProject/
|
├─ Assets/           ← dein eigentlicher Projektinhalt
├─ Packages/         ← Abhängigkeiten / Unity-Packages
├─ ProjectSettings/  ← globale Unity-Projekteinstellungen
|
├─ Library/          ← generierter Cache
├─ Temp/             ← temporär
├─ Logs/             ← Logs
├─ UserSettings/     ← lokale Editor-Einstellungen
|
├─ *.sln             ← von Unity generiert
└─ *.csproj          ← von Unity generiert
```

Die `.sln` und `.csproj` sind dabei **nicht die Wahrheit über dein Projekt**. Unity generiert sie hauptsächlich für Rider/Visual Studio/VS Code. Wenn du sie löschst, kann Unity sie wieder erzeugen.

Assets/ – hier spielt sich fast alles ab

Das ist vergleichbar mit `src/` plus Ressourcen plus Level-Dateien plus Konfiguration gleichzeitig.

Darin können zum Beispiel liegen:

```
Assets/
├─ _Demo/
│   └─ BehaviorTreePolice/
│       ├── Scripts/
│       ├── Scenes/
│       ├── Prefabs/
│       ├── Animations/
│       └─ ...
├─ Synty/
├─ Plugins/
└─ ...
```

Und hier ist ein wichtiger Unterschied zu normaler Softwareentwicklung:

Unity behandelt fast alles als Asset.

Ein C#-Script ist ein Asset.

Eine Textur ist ein Asset.

Ein 3D-Modell ist ein Asset.

Eine Scene ist ein Asset.

Ein Animator Controller ist ein Asset.

Ein Behavior Tree kann ein Asset sein.

Deshalb liegt auch so viel unterschiedlicher Kram unter demselben `Assets/`-Ordner.

Scenes sind ungefähr deine Levels / Programmzustände

Eine `.unity`-Datei ist eine **Scene**.

Zum Beispiel:

```
PoliceBT_Homemade.unity
```

Darin ist gespeichert:

```
Player  
Sheriff  
Buildings  
Lights  
Camera  
UI  
Patrol Points  
Jail  
Restricted Area  
...
```

Aber nicht nur visuell.

An jedem Objekt hängen **Components**.

Das ist eines der wichtigsten Unity-Konzepte überhaupt:

```
GameObject  
+  
Components
```

Beispielsweise:

```
Sheriff
├── Transform
├── Animator
├── NavMeshAgent
├── PoliceAIContext
├── PoliceBlackboard
└── PoliceBehaviorTreeRunner
```

Der `Sheriff` selbst ist im Grunde nur ein Container.

Die eigentliche Funktionalität kommt durch seine Components.

Wenn du aus klassischer Software kommst, kannst du dir ein `GameObject` ungefähr als **Entity/Objekt mit komponierten Modulen** vorstellen.

Und deine C#-Klassen werden Components

Wenn du schreibst:

```
public class PoliceBehaviorTreeRunner : MonoBehaviour
{
}
```

kann Unity diese Klasse als Component an ein `GameObject` hängen.

Dann existiert in der Scene beispielsweise:

```
Sheriff
  PoliceBehaviorTreeRunner
```

Unity serialisiert dann auch die konfigurierten Felder.

Zum Beispiel:

```
[SerializeField]
private PoliceAIContext context;
```

Im Unity Inspector kannst du dann ein anderes Objekt bzw. eine Component dort hineinziehen.

Das ist extrem wichtig, weil Unity-Projekte dadurch **nicht nur aus Code bestehen**.

Ein Teil deiner Programmlogik steckt im C#-Code.

Ein anderer Teil steckt in der Scene:

Welches Script hängt an welchem Objekt?
Welche Referenz zeigt wohin?
Welche Werte wurden im Inspector eingestellt?

Deshalb kann ein Unity-Projekt perfekt kompilieren und trotzdem zur Laufzeit komplett kaputt sein.

Prefabs

Ein Prefab ist grob gesagt eine **wiederverwendbare GameObject-Vorlage**.

Statt fünf Sheriffs jedes Mal neu zusammenzubauen:

```
Sheriff
├── Model
├── Animator
├── NavMeshAgent
├── Scripts
└── Einstellungen
```

machst du:

```
Sheriff.prefab
```

und platzierst davon Instanzen in Scenes.

Vergleichbar mit einer vorkonfigurierten Objektdefinition.

Dabei kann eine Scene-Instanz einzelne Werte des Prefabs überschreiben.

Warum überall **.meta** -Dateien herumliegen

Das ist etwas, das du unbedingt verstehen solltest.

Neben:

```
PoliceBehaviorTreeRunner.cs
```

liegt:

```
PoliceBehaviorTreeRunner.cs.meta
```

In der `.meta` steckt unter anderem eine **GUID**.

Zum Beispiel sinngemäß:

```
guid: abc123...
```

Unity sagt dann nicht:

„Sheriff verwendet `Assets/Scripts/PoliceBehaviorTreeRunner.cs`“

sondern vereinfacht:

„Sheriff verwendet Asset mit GUID `abc123`.“

Dadurch können Assets umbenannt oder verschoben werden, ohne dass sofort alle Referenzen kaputtgehen.

⚠️ Deshalb niemals `.meta` -Dateien einfach ignorieren oder wahllos löschen.

In Git gehören die mit rein.

Wenn Script und `.meta` auseinandergerissen werden, bekommst du sehr gerne diese wunderbaren Unity-Probleme:

```
Missing Script
```

Packages/

Das ist ungefähr dein:

```
package.json  
pom.xml  
requirements.txt  
NuGet dependencies
```

je nachdem, aus welcher Welt du kommst.

Wichtig ist vor allem:

```
Packages/manifest.json
```

Da steht beispielsweise:

```
"com.unity.inputsystem": "1.19.0",  
"com.unity.ai.navigation": "2.0.12"
```

Unity lädt diese Packages über seinen **Unity Package Manager (UPM)**.

Einige Packages kommen direkt von Unity, andere von GitHub:

```
https://github.com/...
```

Bei dir liegt zum Beispiel auch dein Unity-MCP so im Package-System.

Aber Third-Party-Tools liegen nicht zwingend unter Packages/

Und das macht Unity-Projekte gerne etwas chaotisch.

Game Creator liegt bei dir beispielsweise ungefähr hier:

```
Assets/Plugins/GameCreator/
```

obwohl es konzeptionell ebenfalls eine Dependency ist.

Das liegt daran, dass Asset-Store-Produkte und ältere Unity-Erweiterungen häufig einfach Dateien in Assets/ importieren.

Du kannst also haben:

```
Packages/  
  Unity Package Manager Sachen  
  
Assets/Plugins/  
  Third-Party-Tools  
  
Assets/Synty/  
  gekaufte/importierte Art Assets
```

Alles drei sind externe Dependencies, aber technisch unterschiedlich eingebunden.

ProjectSettings/

Hier stehen Einstellungen für **das gesamte Projekt**.

Zum Beispiel:

```
Unity-Version
Input
Physics
Layers
Tags
Graphics
Build Settings
Player Settings
Quality
Editor Settings
```

Diese Dateien solltest du normalerweise ebenfalls committen.

Wenn du beispielsweise einen eigenen Layer namens:

```
RestrictedArea
```

anlegst, kann das dort gespeichert sein.

Library/

Das ist im Wesentlichen ein gigantischer Unity-Cache.

Unity importiert deine Assets und erzeugt daraus intern optimierte Daten.

Das landet größtenteils in:

```
Library/
```

 **Das gehört normalerweise nicht in Git.**

Du kannst den Ordner grundsätzlich löschen. Unity baut ihn beim nächsten Öffnen neu auf.

Dasselbe gilt meist für:

```
Temp/
Logs/
obj/
```

Noch ein wichtiges Konzept: Unity serialisiert deine Welt

Eine Scene ist keine klassische C#-Datei.

Sie enthält serialisierte Informationen wie:

```
GameObject Sheriff
Transform XYZ
Component NavMeshAgent
Component PoliceBehaviorTreeRunner
Reference → PoliceBlackboard
Reference → Player
...
```

Unity speichert Scenes und Prefabs heutzutage meistens als YAML-artigen Text.

Darum konntest du bzw. Codex Teile deiner Scene theoretisch sogar direkt aus Dateien untersuchen.

Aber:

 **Scenes manuell als Text bearbeiten ist meistens keine gute Idee.**

Besser über den Unity Editor bzw. vernünftige Editor-Tools/MCP.

Wo läuft dein Code eigentlich?

Unity hat einen sogenannten Game Loop.

Vereinfacht ruft Unity Methoden deiner `MonoBehaviour`s auf:

```
Awake()
Start()

Update()
Update()
Update()
...

OnDestroy()
```

Du hast also normalerweise nicht:

```
static void Main()
```

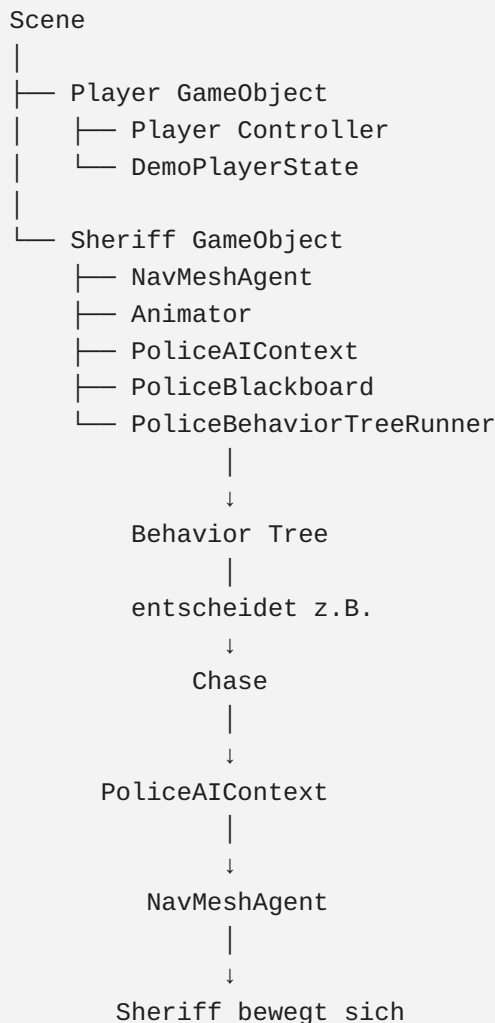
und kontrollierst den gesamten Programmfluss selbst.

Unity besitzt den Hauptloop und ruft deinen Code auf.

Das ist wahrscheinlich eine der größten Umstellungen, wenn du aus klassischer Anwendungsentwicklung kommst.

Dein Behavior-Tree-Projekt konkret

Bei dir sieht die Logik ungefähr so aus:



Das ist eigentlich schon normale Softwarearchitektur. Unity liefert dir lediglich die Laufzeitumgebung und bindet deine Klassen als Components an Scene-Objekte.

Warum deine bearbeitete Synty-Scene problematisch ist

Du hast vermutlich ungefähr:

```
Assets/Synty/PolygonWestern/Scenes/Demo.unity
```

genommen und direkt verändert.

Technisch funktioniert das.

Architektonisch ist es aber unsauber.

⚠ **Denn Assets/Synty/... sollte gedanklich Vendor-Code/Vendor-Assets sein.**

Vergleichbar damit, dass du direkt in:

```
node_modules/
```

oder einer importierten Library herumeditierst.

Nicht exakt dasselbe technisch, aber dasselbe Architekturproblem.

Sauber wäre:

```
Assets/Synty/PolygonWestern/  
  ↳ Original unverändert  
  
Assets/_Demo/BehaviorTreePolice/  
├─ Scenes/  
│   └─ PoliceBT_Homemade.unity  
├─ Scripts/  
├─ Prefabs/  
├─ Animations/  
└─ UI/
```

Das ist genau das Aufräumen, das du jetzt vorhast.

Das wichtigste mentale Modell

Wenn du dir nur **eine Sache** merkst:

```
Unity Project  
├─ Code  
├─ Assets  
├─ Scenes  
│   └─ GameObjects  
│       └─ Components  
├─ Prefabs  
├─ Packages  
└─ Project Settings
```

Und die Verbindungen dazwischen werden überwiegend von Unity **serialisiert und über GUIDs gespeichert**.

Das ist dein Projekt.

Nicht die `.sln`.

Nicht die `.csproj`.

Und auch nicht nur dein C#-Code.

Wenn du dieses Modell verstanden hast, wird dein bestehendes Projekt erheblich weniger „magisch“.